

Implementation & Deployment Blueprint

Supabase Postgres + Upstash Redis + Render Services + Venue-Proof Fallback

PRIMARY DATABASE Supabase Postgres	MEMORY & STREAMS Upstash Redis (TLS)	APP HOSTING Render (Web, API, Worker)	VENUE-PROOF FALLBACK Local Docker / Windows PG 18
---------------------------------------	---	--	--

The Fundamental Architecture Law

PostgreSQL (Supabase) is the final authority for inventory correctness. Redis coordinates work, enforces the Token Bucket shaper, guarantees strict First-Come-First-Served (FCFS) sub-second locks, and buffers contention, but a Redis lock or stream message must not be the only protection against selling the same seat twice. Relational unique constraints guarantee zero double-bookings permanently.

1. Target Cloud Architecture & Component Topologies

Component	Target Platform	Deployment Mode & Runtime	Responsibilities & Failure Boundaries
Durable Database	Supabase Postgres	Managed PostgreSQL 16/17. Connection Pooler (Supervisor) on Port 6543. Direct Connection on Port 5432.	Durable source of truth for inventory, holds, reservations, idempotency keys, and audit ledgers. Unique database constraint guarantees zero double-bookings.
Memory Broker & Bus	Upstash Redis	Serverless Redis (TLS enabled). Redis Streams & Consumer Groups.	Short-lived coordination, Adaptive Token Bucket rate limiting, sub-second holding locks, Redis Stream event delivery (ticketwala:events), and pending message recovery.
API Service	Render Web Service	Docker Web Service (Node.js/Fastify). Auto-scaling container.	Validates requests, enforces idempotency, executes atomic Lua scripts, exposes /health/live , /health/ready , and metrics endpoints.
Background Worker	Render Worker	Docker Background Worker. Independent process.	Consumes Redis Streams via consumer group ticketwalla_workers , executes idempotent SQL upserts to Supabase, handles retries, DLQ, and crash recovery.
Customer & Ops UI	Render Web Service / Vercel	Next.js 15 + React + Tailwind CSS. Static/SSR web deployment.	Customer 200-seat booking flow (live 120s timer) + Contention Observatory dashboard showing real-time seat matrix, throughput, and audit proofs.
Load Testing	k6 on Laptop	Headless CLI Load Runner.	Runs 5,000+ request contention burst locally by default to avoid burning cloud free-tier quotas; connects to API under controlled plan.
Venue Fallback	Local Docker / Windows Stack	Local PostgreSQL 18 + Local Redis.	Guarantees 100% demo success even if hackathon venue Wi-Fi crashes or cloud services sleep.

2. Supabase Connection Strategy & Secret Management

Supabase Dual Connection Mode

- Transaction Pooler URL (Port 6543 via Supavisor):**
Used by `apps/api`. Under burst traffic (5,000+ requests), Fastify worker threads pool queries through Supavisor, preventing `sorry, too many clients already` connection exhaustion.
`DATABASE_URL=postgres://postgres:[pass]@aws-[region].pooler.supabase.com:6543/postgres?sslmode=require`
- Direct Connection URL (Port 5432):**
Used by `apps/worker` and migration scripts (`npm run db:migrate`) where transaction-level prepared statements or advisory locks are required.
`DATABASE_DIRECT_URL=postgres://postgres:[pass]@db.[ref].supabase.co:5432/postgres?sslmode=require`

Production Environment Matrix (.env.production)

```
# App & Host
PORT=8000
APP_ENV=production
LOG_LEVEL=info
CORS_ORIGINS=https://ticketwala.onrender.com

# Supabase Postgres
DATABASE_URL=postgres://postgres:[ref]:[pass]@aws-[region].pooler.supabase.com:6543/postgres?sslmode=require
DATABASE_DIRECT_URL=postgres://postgres:[pass]@db.[ref].supabase.co:5432/postgres?sslmode=require
DATABASE_POOL_MIN=2
DATABASE_POOL_MAX=15

# Upstash Redis
REDIS_URL=rediss://default:[token]@[endpoint].upstash.io:6379
REDIS_MAX_RETRIES=5
REDIS_CONNECT_TIMEOUT_MS=5000

# Adaptive Balancing & Invariants
DEFAULT_EVENT_CAPACITY=200
BASE_TOKEN_BUCKET_CAPACITY=1000
BASE_REFILL_RATE_PER_SEC=500
RESERVATION_TTL_SECONDS=120
IDEMPOTENCY_TTL_SECONDS=600
WORKER_BATCH_SIZE=50
WORKER_CONCURRENCY=2
```

3. Missing Production Capabilities Added to the Plan

To adhere strictly to the **Deployment Runbook**, the following critical enterprise safeguards are incorporated into the TicketWala codebase:

- 1. Two-Tier Health Probes: `/health/live` and `/health/ready`**

 - `GET /health/live (Process Liveness)`: Returns immediate `200 OK` if Fastify event loop is responsive. Used by Render orchestrator to restart crashed containers.
 - `GET /health/ready (Dependency Readiness)`: Performs shallow, non-blocking probes against Upstash Redis (`PING`) and Supabase Postgres (`SELECT 1`) with strict 1,500ms timeouts. Returns `503 Service Unavailable` if dependencies are down. Never causes side effects.
- 2. Graceful Shutdown & Zero-Loss Cleanup Protocol**

When Render deploys a new commit or stops a container, it sends `SIGTERM`. TicketWala executes:

 - Stops accepting new incoming HTTP hold requests (returns `503 Shutting Down`).
 - Flushes in-flight Lua script executions.
 - Allows the background worker to finish the current Supabase batch transaction and acknowledge (`XACK`) processed stream events.
 - Closes Upstash Redis client and Supabase PostgreSQL connection pool cleanly.
 - Exits process with code `0` within the 10-second termination grace period.
- 3. Versioned Database Migrations Pipeline**

Rather than executing raw schema scripts randomly at application boot, schema definitions are organized in `db/migrations/` (e.g. `001_init_ticketwala_schema.sql`). Migrations run once during the release phase, completely preventing race conditions between multiple API/worker replicas.

4. The 18-Hour Mission Schedule & Build Order

Mapped directly to the 9 Build Phases from the Runbook, condensed into our strict 18-hour countdown:

Hour Window	Build Phase	Key Tasks & Supabase Integration	Exit Gate Verification
00:00 – 02:00 (120 min)	Build 1 & 3: Scaffolding & Data Model	Create monorepo (`apps/api`, `apps/web`, `apps/worker`). Apply initial schema to Supabase Postgres. Configure Upstash Redis TLS client. Implement `/health/live` & `/health/ready`.	GET /health/ready returns 200 OK with healthy Supabase & Upstash connections.
02:00 – 05:00 (180 min)	Build 2: Transactional Core & Balancing	Write <code>adaptive_balance.lua</code> (Token Bucket), <code>hold_fcfs.lua</code> (FIFO Queue RPOP), <code>confirm.lua</code> , and <code>release_fcfs.lua</code> . Build concurrency race test (50 requests vs 1 seat).	Automated Vitest suite passes: exactly 1 holder, 49 rejected. Zero double-allocations.
05:00 – 08:00 (180 min)	Build 4 & 5: Streams Worker & Recovery	Build consumer group worker (`ticketwalla_workers`). Implement idempotent upsert to Supabase. Implement <code>XAUTOCLAIM</code> crash recovery. Test worker kill & backlog drain.	Worker recovery test passes: 200 events replayed with 0 duplicate rows in Supabase.
08:00 – 11:00 (180 min)	Build 6: Customer Booking Experience	Next.js 15 interactive seat map with configurable capacity N. 120s circular countdown timer. 1-click checkout modal & abandonment release button. QR pass generation.	Customer booking lifecycle (hold → simulated pay → confirmed) operates smoothly against API.
11:00 – 14:00 (180 min)	Build 7 & 8: Observatory & Auditor	Build Contention Observatory dashboard (live seat matrix, RPS ticker, token bucket meter, stream backlog). Build Independent Invariant Auditor (cross-store math check).	Auditor returns 100% green checkmarks across all invariants.
14:00 – 16:00 (120 min)	Build 8: k6 Headless Load Benchmark	Run k6 load test: 5,000 requests against 200 seats (and parameterized 50/1,000 seats). Capture raw JSON/CSV metrics. Run Invariant Auditor immediately post-test.	Raw benchmark captured: 0 oversell, sub-50ms p95 latency, 4,800 clean 409 rejections.
16:00 – 17:30 (90 min)	Build 9: Production Cloud Deployment	Deploy API & Worker Docker services to Render. Connect Supabase & Upstash credentials. Deploy UI. Verify HTTPS endpoints and CORS. Prepare local venue-proof fallback.	Production smoke tests pass: live URLs responsive from mobile devices.
17:30 – 18:00 (30 min)	Release: Rehearsal & Submission	Fill out Demo Handoff Sheet, record 3-minute emergency backup video, rehearse 7-minute pitch, submit repo before cutoff.	Submission completed.

5. Venue-Proof Fallback Strategy (Local Docker / Windows Stack)

Why the Local Fallback is a Non-Negotiable Requirement

Hackathon venue Wi-Fi is notorious for port throttling, captive portals, high latency, and DNS drops. Cloud services on free tiers can also sleep after inactivity. To guarantee an award-winning live demo, TicketWala treats the local stack as a **first-class deployment target**:

- The developer laptop already has **PostgreSQL 18 running locally** as a Windows service (`postgresql-x64-18`).
- We provide a pre-configured `infra/compose.yaml` running local Redis and local Postgres with pre-seeded demo inventory.
- The presentation can be conducted 100% offline on `http://localhost:3000` with zero cloud dependence if venue Wi-Fi fails!

6. Complete Demo Handoff Sheet (Runbook Section 11)

Keep this sheet ready for the judges and final hackathon submission:

Deployment / Evaluation Item	Production Record / Value
Platform Name	TicketWala (High-Contention Flash Reservation & Adaptive Locking Engine)
Production UI URL	https://ticketwala.onrender.com (or Vercel deployed URL)
Production API URL & Health Check	https://api-ticketwala.onrender.com/health/ready
Durable Database	Supabase Postgres (AWS Region, Pooler Port 6543 / Direct Port 5432)
Memory & Event Stream	Upstash Redis (TLS enabled, Redis Streams consumer group ticketwalla_workers)
Git Commit / Release Tag	v1.0.0-release (main)
Database Migration Version	001_init_ticketwala_schema.sql (Applied via reviewed migration pipeline)
Local Venue-Proof Fallback Command	docker compose -f infra/compose.yaml up --build (or npm run dev on local PG 18)
k6 Load Test Profile	5,000 requests, 150 Virtual Users (VUs), 15s duration, strict FCFS assertion
Benchmark Evidence Files	tests/load/results/benchmark_5000_req.json & benchmark_summary.md
Emergency Backup Artifacts	3-Minute Screen Recording walkthrough stored in repository /docs/demo_backup.mp4

7. Final Pre-Deployment Release Gate (Acceptance Checklist)

- ☐ **Gate 1:** All unit and concurrency race tests pass from a clean checkout (npm test).
- ☐ **Gate 2:** Supabase schema migrations pass against a fresh database with all unique constraints intact.
- ☐ **Gate 3:** Concurrency race test proves zero duplicate confirmed allocations for any seat.
- ☐ **Gate 4:** Idempotency tests prove duplicate request keys return cached responses with zero secondary mutations.
- ☐ **Gate 5:** Worker crash test proves safe backlog accumulation in Redis Streams and clean replay into Supabase with zero duplicate records.
- ☐ **Gate 6:** Ingress Token Bucket dynamically shapes traffic and short-circuits sold-out traffic in < 2ms.
- ☐ **Gate 7:** /health/live and /health/ready return 200 OK on production Render containers.
- ☐ **Gate 8:** Headless k6 benchmark sustains 5,000+ requests with zero double-bookings.
- ☐ **Gate 9:** Local venue-proof fallback verified with internet disconnected.